

Adversarial Audit Paper: Ajna ERC-4626 Vaults + CRE Automation

Date: 2026-03-24 Scope: ajna-finance/4626-ajna-vault, ajna-finance/4626-ajna-vault-keeper, 4626fun/convergence-chainlink-hackathon, and official CRE documentation.

1. Executive Summary

The Ajna ERC-4626 vault design is technically coherent but economically liveness-sensitive: users can mint/redeem shares against `totalAssets()`, yet actual exits are hard-gated by buffer liquidity (`maxWithdraw()/maxRedeem()` cap by `BUFFER.total()` in `Vault.sol`). This means safety is not only smart-contract correctness; it critically depends on continuous, correct offchain rebalancing from Ajna buckets back into the buffer.

The biggest user-loss risk is not a classic reentrancy bug; it is valuation-to-liquidity mismatch under stress. `totalAssets()` in `Vault.sol` counts bucket value (`lpToValue`) plus buffer value, but redemptions are paid from buffer only. In auction/bad-debt/debt-lock regimes, assets can be economically impaired or operationally trapped while shares still reference accounting value, creating unfair exits and bank-run ordering.

The biggest operational/liveness risk is the keeper control-plane and config surface (4626-ajna-vault-keeper). Critical behavior depends on `KEEPER_INTERVAL_MS`, oracle mode (`ONCHAIN_ORACLE_PRIMARY/ORACLE_API_URL/FIXED_PRICE`), subgraph availability (`EXIT_ON_SUBGRAPH_FAILURE`), and gas fallback behavior. A wrong-but-valid config can keep the system "running" while reallocating liquidity badly.

CRE improves determinism, replay resistance, and observability in local workspace (`runtimeBridge` idempotency keys, replay nonces, queue dedupe), but current production shape is still mostly an HTTP bridge to a shared hot key (`/cre/keeper/tend|report|sweep`). CRE should be treated as strong orchestration and policy infrastructure, not a magical safety layer. Used correctly, it materially helps; used as a thin wrapper around centralized writes, it mostly shifts failure domains.

2. Repository Map

Repo	Files reviewed	Purpose	Audit relevance
ajna-finance/4626-ajna-vault (latest head, local clone in <code>tmp/audit-sources/4626-ajna-vault</code>)	<code>src/Vault.sol</code> , <code>src/Buffer.sol</code> , <code>src/VaultAuth.sol</code> , <code>src/AjnaVaultLibrary.sol</code> , <code>interfaces/*</code> , <code>test/*</code> , <code>config/vault-config.example.json</code> , <code>script/Vault.s.sol</code> , <code>README.md</code>	Onchain vault/accounting/liquidity logic	Core asset safety, share integrity, role risk, withdrawal behavior
ajna-finance/4626-ajna-vault-keeper (latest head, local clone in <code>tmp/audit-sources/4626-ajna-vault-keeper</code>)	<code>src/keeper.ts</code> , <code>src/utls/env.ts</code> , <code>src/utls/transaction.ts</code> , <code>src/oracle/*</code> , <code>src/subgraph/poolHealth.ts</code> , <code>src/utls/scheduler.ts</code> , <code>.env.example</code> , <code>README.md</code> , integration tests	Offchain rebalance decision and execution logic	Primary liveness + economic decision engine
4626fun/convergence-chainlink-hackathon (local workspace)	<code>cre/cre-workflows/*</code> , <code>cre/cre-workflows/_shared/*</code> , <code>cre/utls/onchain.ts</code> , <code>frontend/api/_handlers/cre/*</code> , <code>frontend/server/_lib/cre/runtimeBridge.ts</code> , <code>frontend/server/_lib/keeprRegistry.ts</code> , <code>frontend/server/keepr/xmtpQueueExecutor.ts</code> , <code>cre/README.md</code>	CRE orchestration, queueing, runtime sink, execution authority	Determines replay/idempotency, authority boundaries, CRE migration feasibility

Repo	Files reviewed	Purpose	Audit relevance
Chainlink CRE Docs (official)	docs.chain.link/cre/, simulation, limits, consensus, deploy access, service quotas	Runtime guarantees and constraints	Required to judge simulation confidence vs production readiness

3. Implemented System Architecture

Onchain components

- Vault.sol
- ERC-4626 entrypoint (deposit, mint, withdraw, redeem)
- totalAssets() = buffer value + removed collateral value + Ajna bucket values
- maxWithdraw/maxRedeem are buffer-limited.
- Buffer.sol
- Internal quote accounting (total, Mana) with onlyVault and lock.
- VaultAuth.sol
- Admin/keeper/swapper roles and mutable policy (pause, fees, bufferRatio, minBucketIndex).
- AjnaVaultLibrary.sol
- Move primitives + constraints (_checkBufferRatio, _validDestination, role-gating for move paths).

Offchain components

- Legacy Ajna keeper (4626-ajna-vault-keeper)
- Periodic run loop (startScheduler + run)
- Oracle/subgraph checks, bucket selection, move execution.
- CRE workflows (cre/cre-workflows/*)
- vault-keeper, auction-settlement, keeper-queue, runtime-*, payout-integrity, ajna-bucket-manager.
- Vercel bridge handlers
- /api/cre/keeper/*, /api/cre/runtime/*, /api/cre/vaults/active.
- Queue execution path
- keeper_actions queue + xmtQueueExecutor.ts, including canonical CSW path for Ajna rebucket (setMinBucketIndex only).

Data dependencies

- Ajna pool onchain state (LUP, HTP, bucket LP/deposits, bankruptcy time, debt lock).
- Oracle sources:
 - CoinGecko API (ORACLE_API_URL)
 - Chronicle onchain (ONCHAIN_ORACLE_ADDRESS, staleness checks)
 - Optional FIXED_PRICE.
 - Subgraph for unsettled auction discovery (SUBGRAPH_URL).
- CRE runtime storage in Postgres:
 - cre_runtime_records, cre_runtime_decisions, cre_runtime_replay_nonces
 - keeper_actions, keeper_vaults, keeper_vault_automation.

Control/authority map

Component	Critical actions	Authority	Notes
VaultAuth.sol	pause/unpause, role changes, fee/cap/buffer/minBucket changes	admin	Single highest-risk role
AjnaVaultLibrary.moveFromBuffer	move buffer -> pool	keeper only	Enforces role via AUTH.isKeeper(msg.sender)
AjnaVaultLibrary.move / moveToBuffer	move within pool / pool -> buffer	admin or keeper	Central to rebalance and exits
recoverCollateral/returnQuoteToken	collateral unwind/recovery	admin or swapper	Pauses vault via removedCollateralValue path
Keeper wallet (KEEPER_PRIVATE_KEY) via bridge	tend, report, sweep*	bearer token + keeper key	HTTP bridge centralization risk
Ajna canonical CSW path	setMinBucketIndex	creator canonical CSW + embedded EOA owner	Explicit fail-closed admin-match checks

Exit path map

Flow	Happy path	Failure mode
Deposit/mint	User assets -> vault -> buffer accounting; shares minted	If paused/cap/fee config invalid, entry blocked
Withdraw/redeem	Shares burned -> buffer accounting decreases -> assets transferred	If BUFFER.total() insufficient, reverts (hard liveness gate)
Keeper refill	moveToBuffer from Ajna buckets	Fails if out-of-range, debt lock, bad debt, oracle/subgraph issues
Emergency collateral recovery	recoverCollateral -> vault paused -> returnQuoteToken	Operator-dependent recovery; can remain paused until quote returned

Direct answers to Q1–Q5

- Q1 (exact flow)
- Deposit/mint: Vault.deposit/Vault.mint -> _deposit -> BUFFER.addQuoteToken + _fill.
- Withdraw/redeem: Vault.withdraw/Vault.redeem -> _withdraw -> BUFFER.removeQuoteToken + _wash -> transfer to receiver.
- Rebalance: move, moveToBuffer, moveFromBuffer in Vault.sol through AjnaVaultLibrary.
- Emergency: VaultAuth.pause or recoverCollateral path (removedCollateralValue), then returnQuoteToken.
- Q2 (asset location)
- User wallet before entry; vault-controlled balances and Ajna LP exposure after entry; buffer is accounting reserve used for exits.
- Q3 (onchain vs offchain)
- Asset state transitions are onchain; timing and policy execution are offchain (keeper/CRE).
- Q4 (safety-critical for exits)
- BUFFER.total maintenance, keeper liveness, Ajna unwindability, and role/key integrity.

- Q5 (non-custodial vs operator-liveness)
- It is non-custodial in key ownership sense, but user exit safety is materially operator-liveness dependent.

4. Critical Assumptions

Assumption	Where it lives (code/config/offchain)	Why it matters	What breaks if false
Buffer can be refilled quickly enough	Vault.sol + keeper cadence/config	Withdrawals are buffer-only	Redemptions revert; first-exit advantage
totalAssets approximates economic reality	Vault.sol.totalAssets, Ajna state reads	Share pricing fairness	Share value can overstate realizable exits
Keeper picks economically safe buckets	keeper.ts, oracle inputs, OPTIMAL_BUCKET_DIFF	Prevents toxic placement	Wrong bucket concentration and trapped liquidity
Oracle inputs are sane and timely	oracle/*, env mode selection	Drives target bucket index	Persistent misallocation from stale/wrong price
Subgraph bad-debt check is trustworthy	poolHealth.ts, EXIT_ON_SUBGRAPH_FAILURE	Health gate for unsafe pool states	Fail-open can rebalance in unhealthy state
Admin behaves and secures keys	VaultAuth.sol	Full policy/role control	Malicious or mistaken config changes can harm users
Keeper keys are not compromised	.env / bridge handlers	Executes writes	Unauthorized tend/report/sweep writes
Ajna auth admin matches canonical CSW for Ajna automation	ajnaManager.ts, xmtQueueExecutor.ts	Fail-closed canonical execution	Ajna rebucket actions blocked or misrouted
CRE sink idempotency keys are unique	runtimeBridge.ts, runtime workflows	Prevent duplicate actions	Duplicate/omitted actions under replay
Queue status transitions remain atomic	keeprr/actions/_updateStatus.ts	At-most-once practical semantics	Stuck/duplicated operational actions
CRE simulation approximates production	CRE docs	Confidence before deploy	False confidence if DON behavior diverges
Operational runbooks are complete	offchain ops	Fast recovery on incident	Extended liveness outages

5. ERC-4626 Accounting Review

- Q6: totalAssets() in Vault.sol adds $\text{BUFFER.lptToValue}(\text{bufferLps}) + \text{removedCollateralValue} + \sum \text{lptToValue}(\text{bucket})$ and converts WAD to asset decimals. This is accounting value, not guaranteed immediate exit value. It can overstate practical exitability during debt lock/auction/bad debt because user exits are still buffer-only.
- Q7:
 - Fee math uses `_getFee` (ceilDiv) and `_getAssetsWithFee`; previews map through fee-adjusted paths (previewDeposit, previewRedeem).
 - `_decimalsOffset()` is correctly overridden (18 - assetDecimals) in Vault.sol, fixing known non-18 decimal share scaling risk.
 - Classic first-depositor inflation attack is reduced by OZ virtual share/asset mechanics; donation-based manipulation is dampened because direct token donations to vault are not directly included in buffer accounting.
- Q8: Mismatch exists by design:

1) assets counted in totalAssets, 2) assets liquid now (buffer), 3) assets only liquid after keeper/market operations (Ajna buckets), 4) assets potentially impaired/trapped by Ajna states.

- Q9: Yes, users can get economically unfair outcomes. Shares can price off accounting while actual exit path is constrained by buffer + keeper timing.
- Q10: Yes. Early withdrawers consume finite buffer; later withdrawers can revert until rebalance. This is material safety/economic risk, not just UX.

6. Buffer and Redemption Risk Review

- Q11: Static bufferRatio is not stress-adaptive; sufficiency depends on redemption velocity, cadence, and unwindability.
- Q12: Buffer depletes; maxWithdraw/maxRedeem drop with BUFFER.total; further withdrawals revert.
- Q13: Yes, structural bank-run dynamic exists.
- Q14: Economic/safety issue: ordering-dependent access to liquidity can force involuntary hold-through-stress.
- Q15: Yes, an attacker can pin/drain buffer through timing and market-state stress.
- Q16: Drift vectors include interest/time drift, fee-on-transfer/non-standard token behavior assumptions, and out-of-band balance changes.

7. Ajna Market-Structure Review

- Q17: Bucket placement controls realized lender economics; accounting may look healthy while practical yield/risk worsens.
- Q18: Keeper assumes LUP/HTP + minBucketIndex gates and skip logic for bad debt/bankruptcy/debt lock.
- Q19: Partially valid under fast moves; lag + data quality can invalidate assumptions quickly.
- Q20: Yes, adverse selection exposure exists as passive liquidity provider.
- Q21: Yes, deterministic optimal-bucket logic can over-concentrate.
- Q22: Yes, strategy can chase yield into fragile or hard-to-exit buckets under stale/mispriced conditions.
- Q23: Yes, liquidity can appear productive while practically trapped.

8. Keeper Risk Review

- Q24: run() gates on pause/bad debt, updates interest, drains, checks in-range/dust/bankruptcy/debt-lock, then rebalances.
- Q25: Critical config: KEEPER_INTERVAL_MS, OPTIMAL_BUCKET_DIFF, BUFFER_PADDING, MIN_MOVE_AMOUNT, MIN_TIME_SINCE_BANKRUPTCY, MAX_AUCTION_AGE, EXIT_ON_SUBGRAPH_FAILURE, ONCHAIN_ORACLE_PRIMARY, ONCHAIN_ORACLE_MAX_STALENESS, FIXED_PRICE, HALT_KEEPER_IF_LUP_BELOW_HTP.
- Q26: Trusted inputs: RPC, subgraph, CoinGecko/Chronicle/fixed price, env config, keeper key custody.
- Q27:
 - offline keeper: no rebalance, buffer decay risk;
 - subgraph fail: fail-open or fail-closed by config;
 - stale/wrong oracle: bad target or abort;
 - fixed misprice: deterministic bad policy;
 - bad debt/live auction: abort;
 - gas estimation fail: default-gas fallback;
 - LUPBelowHTP: optional hard halt.
- Q28: Yes, safe halts exist.

- Q29: Yes, can continue while economically wrong (e.g., fixed misprice, stale accepted inputs, subgraph fail-open).
- Q30: Yes, keeper delays can cause material user harm.
- Q31: 12-hour cadence is high latency risk in volatility.
- Q32: Deterministic path dependency can be exploited around timing.
- Q33: Yes, timing manipulation can induce bad/skip outcomes.
- Q34: Yes, read/decide/write race conditions exist.

9. Oracle and Data Dependency Review

- Q35: Oracle dependence is reintroduced in keeper price path (getPrice) for bucket targeting.
- Q36:
 - CoinGecko/API: centralized/outage/rate-limit risk;
 - Chronicle/onchain: stronger integrity, still staleness risk;
 - fixed price: highest misconfig risk;
 - CRE + Chainlink feeds: strongest integrity if directly wired into deterministic gates.
- Q37: Safest target path is deterministic CRE policy + Chainlink feeds + strict stale/deviation guards.
- Q38: Easiest misconfigure is FIXED_PRICE.
- Q39: Most robust to latency/manipulation is CRE consensus + Chainlink feed path if used as primary policy input.
- Q40: Yes, stale but valid-looking prices can systematically misplace liquidity.
- Q41: Yes, disagreement/drift can cause oscillation and bad rebalances.
- Q42: Yes, stale reference can move vault from safer to fragile bucket.

10. Smart Contract Security Review

- Q43: Privileged roles are admin, keeper, swapper (VaultAuth.sol).
- Q44: Yes, role misuse or compromise can directly or indirectly cause loss.
- Q45:
 - pause/unpause, cap/fee/minBucket updates are admin-controlled;
 - move functions are role-gated through library checks;
 - emergency recovery is operator-mediated.
- Q46: Reentrancy locks exist; primary residual risks are stale-state economics and offchain timing, not classic reentrancy.
- Q47: Hidden assumptions exist in buffer-ratio checks and decimal conversions under dynamic conditions.
- Q48: Yes, contracts assume Ajna valuation proxies that may diverge from immediate realizability.

11. Attack Scenarios

Scenario	Preconditions	Exploit path	Impact	Detectability	Mitigation	Severity
49. Share price manipulation	Thin liquidity + timing edge	Time deposits/redeems around stale accounting and buffer asymmetry	Fairness distortion	Medium	faster cadence, anti-MEV pathing, liquidity-aware previews	Medium
50. Donation/inflation attack	Direct transfer attempts	Donate assets to skew accounting	Low practical exploitability in this design	High	maintain donation-invariant tests	Low

Scenario	Preconditions	Exploit path	Impact	Detectability	Mitigation	Severity
51. Sandwich around deposit/redeem	Public mempool	Front-run keeper/market state then user op	User execution fairness loss	Medium	private orderflow/jitter	Medium
52. Strategic buffer exhaustion	Large/coord exits	Drain buffer before stress	Exit liveness failure, ordering unfairness	High	dynamic buffer + emergency refill playbook	Critical
53. Keeper timing exploitation	Predictable cadence	Shift state near runs to induce skips/bad moves	Economic drag	Medium	event-driven triggers + randomized windows	High
54. Wrong-bucket via stale oracle	Stale accepted input	Deterministic mis-targeting	Yield loss + trap risk	Medium	strict freshness/deviation checks	High
55. Dust-state grieving	Many tiny buckets	Force skip-heavy behavior	Operational drag	High	periodic dust cleanup policy	Medium
56. Borrower toxic flow	Informed borrowers	Borrow against passive placement	MTM and realized losses	Medium	conservative targeting policy	High
57. Auction/bankruptcy trap	Liquidation/bad debt state	Liquidity remains where exits constrained	Severe liveness/economic harm	High	fail-closed stress policy	Critical
58. Operator key compromise	Keeper key leak	Unauthorized bridge writes	Operational and economic damage	Medium-High	HSM/MPC signer + rotation/runbooks	High
59. Misconfigured buffer ratio	Admin/config error	Ratio set too low/high	Exit failures or excess yield drag	High	bounded config guardrails + staged rollout	High
60. Misconfigured fixed price	Human error	Wrong fixed price accepted	Systematic bad rebalancing	High	disable in prod or require strict controls	High
61. CRE workflow bug repeat/skip	Workflow defect	Duplicate/omitted operations	Liveness and consistency failures	Medium	invariant tests + canary deployment	High
62. AI advisory misinfluence	AI output over-trusted	Human follows wrong advisory	Process error; low direct code risk now	High	keep AI non-authoritative	Low
63. Replay/duplication in CRE flows	Retries/duplicate triggers	Same intent posted multiple times	Mostly mitigated by idempotency; residual race risk	Medium	strict sink/executor idempotency	Medium
64. Deterministic vs AI divergence	Conflicting outputs	Operator follows AI over checks	Delayed/incorrect response	Medium	deterministic precedence in runbooks	Informational

12. CRE Design Review

- Q65: Full replacement is premature; best near-term is CRE-led scheduling/policy with narrowly scoped deterministic execution.
- Q66: Best CRE candidates are scheduling, monitoring, deterministic policy checks, queueing/deduping, checkpointing, and alerting.
- Q67: Must remain deterministic/minimal: action construction, auth checks, idempotency, owner verification, allowlists.
- Q68: Never rely on AI for write auth, safety gating, liquidation-sensitive actions, or emergency actions.
- Q69: CRE should be scheduler + monitor + deterministic policy engine + constrained tx orchestrator + HITL escalation.
- Q70:
 - reusable: runtime-*, keep-queue, sink/idempotency, registry filtering;
 - demo-grade aspects: heavy HTTP bridge dependence and prototype native-write fallback pathing;
 - hardening needed: key management, stricter auth defaults, invariant/chaos testing.
- Q71: CRE adds real value (determinism, replay protection, observability) but also complexity.
- Q72: Key CRE risks are misconfig, secret handling, trigger duplication, HTTP dependency, persistence mismatch, write authority, and early-access maturity risk.
- Q73:
 - simulation: strong but not production-equivalent DON behavior;
 - production readiness: depends on deployment hardening and quotas;
 - institutional resilience: requires mature ops controls beyond baseline.

13. Stress Test Outcomes

Stress scenario	Immediate effect	Medium-term effect	User impact	Recovery path	Severity
74. 20/35/50% collateral shocks	LUP/HTP shifts	More skips/lock risk; bad debt at larger shocks	Liveness pressure -> realized impairment risk	stress policy + operator intervention	High/Critical
75. Rapid deleveraging	Utilization shifts quickly	Target bucket stale	Fairness and yield degradation	faster/event-driven loop	High
76. Active liquidation auctions	Keeper may abort	Refill delays	Exit liveness degradation	auction-aware runbooks	High
77. Bad debt emergence	Health gate trips	Prolonged no-rebalance	Severe liveness + economic risk	manual intervention/pool recovery	Critical
78. 25/50/80% TVL redeem pre-rebalance	Buffer exhaustion	Revert windows persist	first-exit advantage	higher dynamic buffer + emergency process	High/Critical
79. Stale oracle/bad price	Wrong target bucket	Repeated misallocation	realized yield loss	strict stale/deviation checks	High
80. Keeper offline 12/24/72h	No actions	Stale positioning compounds	72h can be severe	failover + paging	Medium/High/Critical
81. Subgraph failure	Debt visibility loss	fail-open unsafe continuation possible	latent risk accumulation	fail-closed setting + alerts	Medium/High
82. Admin mistake	Unsafe policy update	Misbehavior under "valid" code	broad immediate risk	multisig/timelock + change control	High

Stress scenario	Immediate effect	Medium-term effect	User impact	Recovery path	Severity
83. CRE workflow outage	Missed schedule windows	queue backlog	mostly liveness	fallback runner + recovery runbook	Medium
84. CRE duplicate trigger	repeated sink attempts	usually deduped, residual races	operational noise/retry churn	stronger dedupe + transition guards	Medium
85. AI advisory nonsense	bad AI verdict text	deterministic checks still pass	low direct risk currently	keep advisory-only	Low

14. Findings

Critical

- Title: Buffer-only exits create structural bank-run liveness risk
- Severity: Critical
- Affected component: Vault.sol + offchain keeper loop
- Evidence: maxWithdraw/maxRedeem cap by BUFFER.total; withdraw/redeem consume buffer path only; keeper must refill via moveToBuffer
- Why it matters: Exit fairness becomes timing-dependent; late users can be locked out during stress
- Exploitability: High under panic, no advanced exploit needed
- Recommended fix: Dynamic buffer policy + faster/event-driven refill + explicit stress controls and user-facing liquidity state
- Title: Accounting value can diverge from realizable exit value
- Severity: Critical
- Affected component: Vault.sol.totalAssets, Ajna exposure
- Evidence: totalAssets sums bucket valuations while withdrawals remain buffer-only
- Why it matters: Shares can look solvent while exits fail or become economically unfavorable
- Exploitability: High in auction/bad-debt/debt-lock windows
- Recommended fix: Add liquidity-aware risk metrics and stronger rebalance SLOs

High

- Title: Oracle/config path can produce systematic wrong rebalances
- Severity: High
- Affected component: keeper oracle stack (oracle/price.ts, env.ts)
- Evidence: CoinGecko/Chronicle/fixed mode switching and fallback behavior
- Why it matters: Wrong inputs drive deterministic wrong bucket choices
- Exploitability: Medium-High
- Recommended fix: disable fixed price in production, enforce freshness/deviation bounds, add source quorum
- Title: Shared keeper wallet via HTTP bridge is concentrated authority
- Severity: High
- Affected component: /api/cre/keeper/*, KEEPR_PRIVATE_KEY
- Evidence: Bridge endpoints execute writes from shared signer
- Why it matters: Key compromise or auth failure affects broad surface
- Exploitability: Medium
- Recommended fix: HSM/MPC signing, scoped route permissions, key rotation + incident runbooks

- Title: Subgraph failure can fail-open on bad-debt gate
- Severity: High
- Affected component: poolHealth.ts, EXIT_ON_SUBGRAPH_FAILURE
- Evidence: Empty-auction fallback unless fail-closed configured
- Why it matters: Keeper may continue with degraded safety visibility
- Exploitability: Medium
- Recommended fix: production fail-closed + immediate alerting
- Title: Keeper cadence/default latency too slow for stress
- Severity: High
- Affected component: KEEPER_INTERVAL_MS defaults and scheduler
- Evidence: 12h default in keeper repo
- Why it matters: liquidity/risk drift between runs
- Exploitability: High via market speed
- Recommended fix: shorter cadence + event triggers + bounded jitter

Medium

- Title: Deterministic target bucket logic is gameable around timing
- Severity: Medium
- Affected component: keeper.ts (optimalBucket = priceIndex + diff)
- Evidence: simple deterministic offset and predictable schedule
- Why it matters: adversaries can shape pre/post-run state
- Exploitability: Medium
- Recommended fix: richer policy and randomized execution windows
- Title: Read-decide-write race remains in keeper and CRE loops
- Severity: Medium
- Affected component: offchain orchestration
- Evidence: separate read/decision/transaction stages
- Why it matters: stale actions can become wrong actions
- Exploitability: Medium
- Recommended fix: pre-submit revalidation + postcondition checks
- Title: CRE adds resilience but also complexity
- Severity: Medium
- Affected component: runtime-*, bridge, DB schema coupling
- Evidence: replay/idempotency controls with cross-system dependencies
- Why it matters: new failure planes
- Exploitability: Medium (operational)
- Recommended fix: hardening, chaos tests, canary + rollback flow

Low

- Title: Administrative centralization lacks visible governance hardening evidence
- Severity: Low
- Affected component: VaultAuth.sol
- Evidence: broad onlyAdmin mutability
- Why it matters: human/admin key error risk
- Exploitability: Medium via process weakness
- Recommended fix: multisig + timelock + alerts

- Title: Recovery path is operator-dependent
- Severity: Low
- Affected component: recoverCollateral/returnQuoteToken
- Evidence: pause requires trusted restitution flow
- Why it matters: recovery quality drives downtime
- Exploitability: Low direct, medium operational
- Recommended fix: tested runbooks and bounded SLAs

Informational

- Title: AI advisory path is non-authoritative (good pattern)
- Severity: Informational
- Affected component: /api/cre/keeper/_aiAssess.ts, payout-integrity/main.ts
- Evidence: deterministic fallback verdict and advisory use only
- Why it matters: preserves deterministic safety boundary
- Exploitability: N/A
- Recommended fix: preserve this invariant

15. Recommended Architecture

Choice: C. Move scheduling/decisioning to CRE but keep onchain execution narrowly scoped.

Rationale:

Core risk is policy timing + data quality + liveness (CRE is strong here).

Safety-critical writes should remain minimal, deterministic, and allowlisted.

Current local stack already includes strong building blocks: runtime idempotency, queue dedupe, canonical CSW checks.

Migration shape:

CRE handles schedule, monitoring, deterministic policy synthesis, and queueing.

Hardened execution layer performs allowlisted writes with strict preflight checks.

AI remains advisory-only.

Legacy keeper path remains fallback during migration, then retired after parity/SLO proof.

16. Production Readiness Verdict

- smart contract safety: 7/10
- accounting integrity: 6/10
- redemption fairness: 4/10
- liveness robustness: 4/10
- oracle/data robustness: 5/10
- operator risk: 4/10
- CRE suitability: 6/10
- overall deployability: 5/10
- Would this be deployed with real user funds today?

Only in a constrained pilot, not mainnet scale.

- Under what strict conditions only?
- conservative TVL caps;
- higher dynamic buffer targets;

- faster/event-driven keeper cadence;
 - fail-closed production settings on critical dependencies;
 - hardened signer custody and tested incident runbooks.
 - Minimum blockers before mainnet scale
- Buffer-liveness hardening with explicit stress policy.
- Oracle/data policy hardening (no unsafe fixed-price operations).
- Operator key and governance hardening (multisig/timelock/HSM).
- Proven CRE+queue reliability under replay/outage/duplication tests.
- User-facing disclosure of realizable exit vs accounting value.

17. Missing Information / Next Artifacts Needed

- deployed addresses per active vault and environment
- exact production vault config JSONs
- production keeper env/config values (KEEPER_INTERVAL_MS, oracle mode, fail-open/closed flags)
- formal pool/bucket selection policy and approved risk limits
- admin/keeper/swapper ownership model (multisig/timelock/signer inventory)
- CRE workflow IDs, deployment targets, and secret custody model
- incident runbooks for buffer depletion, oracle outage, key compromise, CRE outage
- SLO/SLI targets for rebalance latency and redemption availability
- postmortem and escalation templates